Module 14 — Minimum Spanning Trees

Sources: CLRS 4e ch. 21 (Minimum Spanning Trees) · Skiena 3e §8.1 (Minimum Spanning Trees, incl. Prim, Kruskal, union–find, variations)

Big Idea

Problem. Given a connected, undirected graph $G = (V, E)$ with weights $w : E \to \mathbb{R}$, find an acyclic subset $T \subseteq E$ connecting all vertices and minimizing $w(T) = \Sigma\, w(u,v)$.

(CLRS's motivating story is circuit design: interconnect n pins with $n-1$ wires using the least total wire. Skiena's is broader: "Minimum spanning trees are the answer whenever we need to connect a set of points… cheaply using the smallest amount of roadway, wire, or pipe.")

Since T is acyclic and spans, it is a **tree** — and by Theorem B.2, *every* spanning tree has exactly $|V| - 1$ edges. So *"there is no point in minimizing the number of edges"*; the whole problem is minimizing **weight**. Skiena puts it well: *"Any tree is the smallest possible connected graph in terms of number of edges, but the minimum spanning tree is the smallest connected graph in terms of edge weight."*

The remarkable fact is that greedy works — and that it works in two completely different ways. Both classical algorithms are instances of one generic scheme:

```
GENERIC-MST(G, w)
1  A = ∅
2  while A does not form a spanning tree
3      find an edge (u,v) that is safe for A
4      A = A U {(u,v)}
5  return A
```

→ **C++ implementation:** A1 GENERIC-MST

with **loop invariant: A is a subset of some minimum spanning tree.** An edge is **safe** if adding it preserves that invariant. Everything in this module reduces to one theorem — **the cut property (Theorem 21.1)** — which tells you how to recognize a safe edge, and then two ways of applying it:

	KRUSKAL	PRIM
What `A` looks like	a **forest** of many trees	a **single tree**
Safe edge chosen	cheapest edge joining **any two** components	cheapest edge joining **the tree** to a non-tree vertex
Enabling data structure	**union–find** (M10)	**min-priority queue** (M05)
Resembles	connected components (M13)	Dijkstra (M15)

Remember months later: *the cut property is the whole subject — a lightest edge crossing any cut that respects* `A` *is safe. Kruskal applies it by sorting edges and using union–find; Prim applies it by growing one tree with a heap. Both* `O(E lg V)`*. Every MST is also a minimum-bottleneck spanning tree, and the MST is unique iff… well, distinct weights suffice.*

What You Should Be Able To Do After This Chapter

- State the cut property precisely — **cut**, **respects**, **light edge**, **safe edge** — and prove Theorem 21.1 with the cut-and-paste exchange argument.
- Derive both algorithms as corollaries of the cut property (Corollary 21.2).
- Write Kruskal and Prim from memory, and pick the right one from the graph's density.
- Give both $O(V^2)$ and $O(E \lg V)$ Prim, and know when the first one wins.
- State the **cycle property** and use it (Exercise 21.1-5).
- Explain why every MST is a **minimum-bottleneck** spanning tree, and why the converse fails.
- Adapt MST to maximum spanning trees, minimum-product spanning trees, and bottleneck problems by transforming the weights.
- Say what MST **cannot** do: Steiner trees and low-degree spanning trees.
- Name Borůvka, and know that near-linear and randomized-linear MST algorithms exist.

Part 1 — The Cut Property (CLRS 21.1)

Definitions — get these exactly right

- A **cut** `(S, V − S)` is a **partition of** `V`.
- An edge `(u,v)` **crosses** the cut if one endpoint is in `S` and the other in `V − S`.
- A cut **respects** a set `A` of edges if **no edge of** `A` **crosses it**.
- An edge is a **light edge** crossing a cut if its weight is minimum among all edges crossing that cut. *(There can be several, in case of ties.)*
- An edge is **safe for** `A` if `A ∪ {(u,v)}` is still a subset of some MST.

Theorem 21.1 (the cut property)

> *Let `A ⊆ E` be included in some minimum spanning tree, let `(S, V−S)` be **any cut that respects** `A`, and let `(u,v)` be a **light edge crossing that cut**. Then `(u,v)` is **safe for** `A`.*

Proof (cut-and-paste — the same exchange argument as M12). Let `T` be an MST containing `A`. If `(u,v) ∈ T`, done. Otherwise, adding `(u,v)` to `T` creates a cycle with the unique simple path `p` from `u` to `v` in `T`. Since `u` and `v` are on opposite sides of the cut, **at least one edge `(x,y)` of `p` also crosses the cut.** That edge is not in `A` (the cut respects `A`). Removing `(x,y)` splits `T` in two; adding `(u,v)` reconnects them:

```
T′ = (T − {(x,y)}) ∪ {(u,v)}
```

Since `(u,v)` is light and `(x,y)` also crosses the cut, `w(u,v) ≤ w(x,y)`, so

```
w(T′) = w(T) − w(x,y) + w(u,v) ≤ w(T)
```

But `T` is minimum, so `w(T) ≤ w(T′)` — hence `T′` is also an MST. Finally `A ⊆ T′` (because `A ⊆ T` and `(x,y) ∉ A`), so `A ∪ {(u,v)} ⊆ T′`, making `(u,v)` safe. ∎

Read the shape of that proof once more. *Take any optimum, swap your greedy choice in for a comparable element, show the result is no worse.* Identical to Theorem 15.1 and Huffman's Lemma 15.2 (M12), and to Skiena's own Prim/Kruskal proofs below.

Corollary 21.2 — the form both algorithms actually use

> *Let `A` be included in some MST, and let `C = (V_C, E_C)` be a **connected component (tree) of the forest `G_A = (V, A)`**. If `(u,v)` is a **light edge connecting `C` to some other component**, then `(u,v)` is safe for `A`.*

Proof. The cut `(V_C, V - V_C)` respects `A`, and `(u,v)` is a light edge for it. ∎

Why `GENERIC-MST` terminates in exactly `|V| - 1` iterations: `A` is always acyclic (a subset of a tree), so `G_A` is a forest; a safe edge always joins two distinct components; each iteration reduces the component count by exactly one, starting from `|V|`.

The companion: the cycle property

Exercise 21.1-5. *Let `e` be a maximum-weight edge on some cycle of `G`. Then there is an MST of `G - {e}` that is also an MST of `G`* — i.e. **the heaviest edge on any cycle is dispensable.**

The cut property tells you what to **include**; the cycle property tells you what to **exclude**. Kruskal is the cut property applied greedily forward; the "reverse-delete" algorithm (delete the heaviest edge whose removal keeps the graph connected) is the cycle property applied greedily backward, and it also produces an MST.

Verified: on 300 random graphs, for **every one of the `2ⁿ - 2` cuts**, forcing the light edge into the tree and completing greedily produced exactly the MST weight (the cut property); and deleting any edge that is the *strict* maximum on some cycle never changed the MST weight (the cycle property).

Two useful facts about uniqueness and ties

> ***Skiena:*** *"The minimum spanning tree of a graph is unique if all `m` edge weights in the graph are distinct. Otherwise the order in which Prim's/Kruskal's algorithm breaks ties determines which minimum spanning tree is returned."*

*(The converse is false — Exercise 21.1-6. And Exercise 21.1-8 gives the more refined statement: **all MSTs of a graph have the same sorted list of edge weights**, even when they differ as edge sets.)*

Verified: CLRS's Figure 21.1 graph has MST weight 37 and is **not** unique — the second-best spanning tree also weighs 37 (swap `(b,c)` for `(a,h)`). With all weights distinct, the MST was unique in 200/200 random graphs.

Part 2 — Kruskal's Algorithm

Idea. Consider edges in increasing weight order; add an edge iff its endpoints are in **different components**. By Corollary 21.2 that edge is a light edge connecting its component to another, hence safe.

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3      MAKE-SET(v)
4  create a single list of the edges in G.E
5  sort the list into monotonically increasing order by weight w
6  for each edge (u,v) taken from the sorted list in order
7      if FIND-SET(u) ≠ FIND-SET(v)
8          A = A U {(u,v)}
9          UNION(u, v)
10 return A
```

→ **C++ implementation:** A2 MST-KRUSKAL

Skiena's framing, which explains why no cycle check is needed: *"Kruskal's algorithm builds up connected components of vertices... If the endpoints lie in different components, we insert the edge and merge the two components into one. **Since each connected component always is a tree, we never need to explicitly test for cycles.**"*

Skiena's correctness proof — a clean exchange argument in three sentences. Suppose Kruskal first errs at edge `(x,y)`. Inserting `(x,y)` into `T_min` creates a cycle containing the `x`–`y` path. Since `x` and `y` were in **different components** when `(x,y)` was chosen, at least one edge (v_1,v_2) on that path was considered by Kruskal **later** than `(x,y)`, so $w(v_1,v_2) \geq w(x,y)$; exchanging them yields a tree no heavier. ∎

Running time. Sorting dominates: $O(E \lg E)$. The $O(E)$ disjoint-set operations cost $O(E\,\alpha(V))$ with union by rank + path compression (M10), and since $\alpha(V) = O(\lg V)$ this is dominated. Because $|E| < |V|^2$, $\lg|E| = O(\lg V)$, so the total is

$$O(E \lg E) = O(E \lg V)$$

Note what carries the algorithm: *"a clever data structure called union–find can support such queries in $O(\lg n)$ time... Observe again the impact that the right data structure can have when implementing a straightforward algorithm."* Without it, testing connectivity by BFS/DFS each time gives $O(mn)$.

C++ Implementation

```
#include <algorithm>
#include <climits>
#include <numeric>
#include <utility>
#include <vector>

struct Edge {
    int u, v;
    long long w;
};

class WeightedGraph {
public:
    explicit WeightedGraph(int n) : adj_(n) {}

    void addEdge(int u, int v, long long w) {
        adj_[u].push_back({v, w});
        adj_[v].push_back({u, w});
        edges_.push_back({u, v, w});
    }

    int n() const { return (int)adj_.size(); }
    const vector<Edge>& edges() const { return edges_; }
    const vector<pair<int, long long>>& adj(int u) const { return
adj_[u]; }

private:
    vector<vector<pair<int, long long>>> adj_;
    vector<Edge> edges_;
};

// ---------------- disjoint sets (M10), union by rank + path
compression
class DisjointSet {
public:
    explicit DisjointSet(int n) : p_(n), rank_(n, 0) {
iota(p_.begin(), p_.end(), 0); }
    int find(int x) {
        int root = x;
        while (p_[root] != root) root = p_[root];
```

```
        while (p_[x] != root) { const int next = p_[x]; p_[x] =
root; x = next; }
        return root;
    }
    bool unite(int x, int y) {
        int a = find(x), b = find(y);
        if (a == b) return false;
        if (rank_[a] > rank_[b]) swap(a, b);
        p_[a] = b;
        if (rank_[a] == rank_[b]) ++rank_[b];
        return true;
    }
private:
    vector<int> p_, rank_;
};

struct MstResult {
    long long weight = 0;
    vector<Edge> tree;
    bool connected = false;
};

// Safe edge = lowest-weight edge joining two distinct components
(Corollary 21.2).
MstResult kruskal(const WeightedGraph& g) {
    vector<Edge> e = g.edges();
    sort(e.begin(), e.end(), [](const Edge& a, const Edge& b) {
return a.w < b.w; });
    DisjointSet ds(g.n());
    MstResult r;
    for (const auto& x : e)
        if (ds.unite(x.u, x.v)) {                        //
different trees: safe to add
            r.tree.push_back(x);
            r.weight += x.w;
        }
    r.connected = (int)r.tree.size() == g.n() - 1;
    return r;
}
```

Implementation notes. `unite` returning `bool` **is** the "are they in the same component?" test — one call instead of two `find`s plus a `union`. Running Kruskal on a **disconnected** graph is not an error: it produces the **minimum spanning forest**, and `connected` reports which you got. That behavior is free and frequently what you actually want.

Part 3 — Prim's Algorithm

Idea. Grow a **single tree** from an arbitrary root. At each step add the lightest edge crossing the cut (tree vertices, non-tree vertices) — safe by Corollary 21.2.

> ***Skiena's statement:*** *"Prim's algorithm clearly creates a spanning tree, because no cycle can be introduced by adding edges between tree and non-tree vertices. But why should it be of minimum weight over all spanning trees?* ***We have seen ample evidence of other natural greedy heuristics that do not yield a global optimum. Therefore, we must be particularly careful to demonstrate any such claim.***"

Skiena's proof mirrors Kruskal's: suppose Prim first errs by inserting (x,y). There is a path p from x to y in T_{min}, which must use an edge (v_1,v_2) with $v_1 \in T_{prim}$ and $v_2 \notin T_{prim}$. *"This edge (v_1,v_2) must have weight at least that of (x,y), or else Prim's algorithm would have selected it before (x,y) when it had the chance."* Swap them. ∎

```
MST-PRIM(G, w, r)
 1  for each vertex u ∈ G.V
 2      u.key = ∞ ;  u.π = NIL
 4  r.key = 0
 5  Q = ∅ ;  for each u ∈ G.V : INSERT(Q, u)
 8  while Q ≠ ∅
 9      u = EXTRACT-MIN(Q)                    // add u to the tree
10      for each vertex v in G.Adj[u]         // update keys of u's
non-tree neighbours
11          if v ∈ Q and w(u,v) < v.key
12              v.π = u
13              v.key = w(u,v)
14              DECREASE-KEY(Q, v, w(u,v))
```

→ **C++ implementation:** A3 MST-PRIM

The three-part loop invariant:

1. $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$;

2. the vertices already in the tree are exactly `V − Q`;
3. for all `v ∈ Q` with `v.π ≠ NIL`, `v.key < ∞` and `v.key` is the weight of a **light edge** `(v, v.π)` connecting `v` to a tree vertex.

Skiena's implementation insight is the same one, stated operationally: *"It keeps track of the cheapest edge linking every non-tree vertex in the tree… since the most recently inserted vertex is the only change in the tree, all possible edge-weight updates must come from its outgoing edges."* **That is why the inner loop only scans `Adj[u]`, and why the total edge work is `Θ(E)` rather than `Θ(VE)`.**

Complexity — three implementations, and when to use each

PRIORITY QUEUE	`EXTRACT-MIN` × `V`	`DECREASE-KEY` × `E`	TOTAL	BEST WHEN
linear array scan (`O(V²)` Prim)	`O(V)` each	`O(1)` each	**`Θ(V²)`**	**dense** graphs, `E = Θ(V²)`
binary min-heap	`O(lg V)` each	`O(lg V)` each	**`O(E lg V)`**	general purpose
Fibonacci heap	`O(lg V)` amortized	**`O(1)` amortized**	**`O(E + V lg V)`**	very large sparse graphs; theory

The `Θ(V²)` version has no heap at all — it's the one Skiena implements — and *"a good illustration of the power of data structures to speed up algorithms"*: the naive "scan all `m` edges each of `n` iterations" is `O(mn)`; keeping the `distance[]`/`parent[]` arrays drops it to `O(n²)`.

Kruskal vs. Prim in practice: *"Kruskal's algorithm… proves more efficient on sparse graphs"* (`O(E lg E)`, and the sort is often the only real cost); dense-Prim at `Θ(V²)` beats `O(E lg V) = O(V² lg V)` on dense graphs. In competitive programming: **sparse ⟹ Kruskal, dense ⟹ `O(V²)` Prim.**

C++ Implementation

```
#include <algorithm>
#include <climits>
#include <functional>
#include <queue>
#include <utility>
#include <vector>
```

```
// Safe edge = lightest edge crossing the cut (tree, non-tree).
Lazy binary heap.
MstResult primHeap(const WeightedGraph& g, int root) {
    const int n = g.n();
    const long long INF = LLONG_MAX / 4;
    vector<long long> key(n, INF);
    vector<int> parent(n, -1);
    vector<char> inTree(n, 0);
    using Item = pair<long long, int>;              // (key, vertex)
    priority_queue<Item, vector<Item>, greater<Item>> q;

    key[root] = 0;
    q.push({0, root});
    MstResult r;
    while (!q.empty()) {
        const auto top = q.top(); q.pop();
        const int u = top.second;
        if (inTree[u]) continue;                        // stale
entry: skip
        inTree[u] = 1;
        if (parent[u] != -1) { r.tree.push_back({parent[u], u,
key[u]}); r.weight += key[u]; }
        for (const auto& e : g.adj(u))
            if (!inTree[e.first] && e.second < key[e.first]) {
                key[e.first] = e.second;
                parent[e.first] = u;
                q.push({e.second, e.first});            // lazy
decrease-key
            }
    }
    r.connected = (int)r.tree.size() == n - 1;
    return r;
}

// Exercise 21.2-2 / Skiena: O(V^2), no heap. Best for dense
graphs.
MstResult primDense(const WeightedGraph& g, int root) {
    const int n = g.n();
    const long long INF = LLONG_MAX / 4;
    vector<long long> key(n, INF);
    vector<int> parent(n, -1);
```

```
    vector<char> inTree(n, 0);
    key[root] = 0;
    MstResult r;
    for (int iter = 0; iter < n; ++iter) {
        int u = -1;
        for (int i = 0; i < n; ++i)                    // linear
scan for the minimum key
            if (!inTree[i] && (u < 0 || key[i] < key[u])) u = i;
        if (u < 0 || key[u] >= INF) break;             // graph is
disconnected
        inTree[u] = 1;
        if (parent[u] != -1) { r.tree.push_back({parent[u], u,
key[u]}); r.weight += key[u]; }
        for (const auto& e : g.adj(u))
            if (!inTree[e.first] && e.second < key[e.first]) {
                key[e.first] = e.second;
                parent[e.first] = u;
            }
    }
    r.connected = (int)r.tree.size() == n - 1;
    return r;
}
```

The "lazy decrease-key" idiom, which is worth understanding once and then reusing forever (it recurs verbatim in Dijkstra, M15). `std::priority_queue` has **no `decrease-key` operation**. Rather than building an indexed heap (M05's `IndexedMinPQ`), just **push a new `(key, vertex)` pair every time the key improves** and discard stale pops with `if (inTree[u]) continue;`. The heap holds `O(E)` entries instead of `O(V)`, so the bound becomes `O(E lg E) = O(E lg V)` — **asymptotically identical**, and far simpler. Use it.

Verified: Kruskal, `primHeap`, `primDense` and Borůvka produce **identical MST weights** on 400 random graphs and on 60-vertex dense graphs, and all four match exhaustive enumeration of every spanning tree. On CLRS's Figure 21.1 instance all four return **37**.

Part 4 — Borůvka's Algorithm

The **oldest** MST algorithm — O. Borůvka, **1926** — and the one that parallelizes.

Idea. In each round, *every* component simultaneously selects **its own cheapest outgoing edge**; add them all and merge. Each edge chosen is safe by Corollary 21.2. Since every component merges with at least one other, **the number of components at least halves each round ⟹ `O(lg V)` rounds**, each `O(E)`, for `O(E lg V)`.

The one catch: with tied weights, two components can each pick the *same* edge, or a cycle of picks can form. **The standard fix is a total order on edges** — break weight ties by edge index — which is what the implementation below does.

```
#include <algorithm>
#include <vector>

// Each round, every component picks its own cheapest outgoing
edge; merge them all.
MstResult boruvka(const WeightedGraph& g) {
    const int n = g.n();
    DisjointSet ds(n);
    MstResult r;
    int components = n;
    while (components > 1) {
        vector<int> best(n, -1);                    // best[c] =
index of c's cheapest edge
        const auto& e = g.edges();
        for (int i = 0; i < (int)e.size(); ++i) {
            const int a = ds.find(e[i].u), b = ds.find(e[i].v);
            if (a == b) continue;
            for (int c : {a, b})
                if (best[c] < 0 || e[i].w < e[best[c]].w ||
                    (e[i].w == e[best[c]].w && i < best[c]))    //
deterministic tie-break
                    best[c] = i;
        }
        bool merged = false;
        for (int c = 0; c < n; ++c) {
            if (best[c] < 0) continue;
            const Edge& x = e[best[c]];
            if (ds.unite(x.u, x.v)) {
                r.tree.push_back(x);
                r.weight += x.w;
                --components;
                merged = true;
            }
```

```
        }
        if (!merged) break;                                    //
disconnected
    }
    r.connected = (int)r.tree.size() == n - 1;
    return r;
}
```

> *Outside / Engineering Context — why Borůvka matters*
>
> ***It's the only one of the three that parallelizes cleanly.*** *Each round's "every component finds its cheapest outgoing edge" is an embarrassingly parallel reduction, and there are only* `O(lg V)` *rounds. Every distributed/GPU MST implementation is Borůvka-based.*
>
> *It is also the base of the* ***asymptotically best*** *algorithms. CLRS's chapter notes track the history: Fredman–Tarjan* `O(E lg* V)`*;* *Gabow–Galil–Spencer–Tarjan* `O(E lg lg* V)`*;* ***Chazelle*** `O(E α(E,V))` *— which "does not follow the greedy method"; Pettie–Ramachandran's provably optimal algorithm (whose exact complexity is unknown, but is optimal); and* ***Karger–Klein–Tarjan's randomized*** `O(V + E)` ***expected-time algorithm****, which uses recursion in the style of median-of-medians (M05) plus Borůvka steps plus King's linear-time* ***MST verification****. So: MST can be done in randomized linear time; whether it can be done in deterministic comparison-based linear time is open.*

Part 5 — Variations (Skiena 8.1.4)

These are the interview-relevant part of the chapter. Three problems that look different and are the same problem with transformed weights, and two that look similar and are much harder.

Solvable by transformation

VARIANT	TRANSFORMATION	WHY
Maximum spanning tree	negate all weights, run MST	*"an evil telephone company is contracted to connect a bunch of houses... paid a price proportional to the amount of wire they install. Naturally, they will seek to build the most expensive possible spanning tree!"* The most negative tree in the negated graph is the maximum tree in the original.
Minimum-product spanning tree (all weights positive)	replace `w` by `lg w`, run MST	`lg(a·b) = lg a + lg b` turns a product into a sum
Minimum-bottleneck spanning tree (minimize the *maximum* edge)	**nothing — every MST already is one**	Follows from Kruskal's correctness: the MST's heaviest edge is the smallest possible

The warning attached to negation is important: *"Most graph algorithms do not adapt so easily to negative numbers. Indeed, shortest path algorithms have trouble with negative weights, and certainly do not generate the longest possible path using this weight negation technique."* **MST is the exception, not the rule** — because the cut property never compares a path to a path, only an edge to an edge.

On bottleneck trees, Skiena adds the simpler alternative: *"A less efficient but conceptually simpler way to solve such problems might be to delete all 'heavy' edges from the graph and ask whether the result is still connected. These kinds of tests can be done with BFS or DFS."* — i.e. binary search on the bottleneck value plus a connectivity check, `O((V+E) lg W)`. **Note the converse fails:** a minimum-bottleneck spanning tree need not be an MST (minimize the max edge and the rest can be arbitrarily bad).

```
#include <algorithm>
#include <climits>
#include <numeric>
#include <vector>

// Maximum spanning tree: negate the weights and run any MST
algorithm.
MstResult maximumSpanningTree(const WeightedGraph& g) {
```

```
    WeightedGraph h(g.n());
    for (const auto& e : g.edges()) h.addEdge(e.u, e.v, -e.w);
    MstResult r = kruskal(h);
    r.weight = -r.weight;
    for (auto& e : r.tree) e.w = -e.w;
    return r;
}

// The bottleneck of a tree: its heaviest edge. Every MST minimises
this.
long long bottleneck(const MstResult& r) {
    long long b = LLONG_MIN;
    for (const auto& e : r.tree) b = max(b, e.w);
    return r.tree.empty() ? 0 : b;
}

// Problem 21-1: the second-best spanning tree differs from the MST
by exactly one
// edge swap, so it suffices to delete each MST edge in turn and
rebuild.
long long secondBestMST(const WeightedGraph& g) {
    const int n = g.n();
    vector<int> idx(g.edges().size());
    iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(),
              [&](int a, int b) { return g.edges()[a].w < g.edges()
[b].w; });

    vector<int> chosen;                              // indices of
the MST's edges
    {
        DisjointSet ds(n);
        for (int i : idx) if (ds.unite(g.edges()[i].u, g.edges()
[i].v)) chosen.push_back(i);
    }
    if ((int)chosen.size() != n - 1) return LLONG_MAX;

    long long second = LLONG_MAX;
    for (int skip : chosen) {
        DisjointSet ds(n);
        long long total = 0;
        int used = 0;
```

```
        for (int i : idx) {
            if (i == skip) continue;
            if (ds.unite(g.edges()[i].u, g.edges()[i].v)) { total
+= g.edges()[i].w; ++used; }
        }
        if (used == n - 1) second = min(second, total);
    }
    return second;
}
```

Note the correctness argument behind `secondBestMST` (CLRS Problem 21-1): the second-best spanning tree differs from the MST in **exactly one edge**, so it suffices to try deleting each of the `|V| - 1` tree edges and re-running. That gives `O(V · E lg V)` here; the intended `O(V²)` solution precomputes `max[u][v]` = the heaviest edge on the tree path between every pair, then takes `min over non-tree (u,v) of w(T) − max[u][v] + w(u,v)`.

Verified: over 400 random graphs, `maximumSpanningTree` matched the maximum over all enumerated spanning trees; `bottleneck(kruskal(g))` matched the minimum-over-all-spanning-trees of the maximum edge; and `secondBestMST` matched the second element of the sorted multiset of all spanning-tree weights.

Not solvable by these techniques

> ***Steiner tree.*** *"Suppose we want to wire a bunch of houses together, but have the freedom to add extra intermediate vertices to serve as a shared junction." Adding Steiner points can only help, and the problem becomes* ***NP-hard*** *(M19, M20).*
>
> ***Low-degree spanning tree.*** *"What if we want to find the minimum spanning tree where the highest degree of a node in the tree is small? The lowest max-degree tree possible would be a simple path… Such a path that visits each vertex once is called a Hamiltonian path" — also* ***NP-hard****.*

The lesson to carry: MST is a rare, beautiful island of tractability. Perturb the objective slightly — add junction points, constrain degrees, ask for the *longest* simple path — and you fall off the cliff into NP-hardness.

Recognition Patterns

PROBLEM SAYS	ANSWER
"connect all of these as cheaply as possible", cabling, piping, roads	**MST**
"cluster these points" / single-linkage hierarchical clustering	**MST** — cutting the `k−1` heaviest MST edges gives the `k` single-linkage clusters
"minimize the maximum edge on a connecting network"	any **MST** (it's automatically a bottleneck tree), or binary search + connectivity
"maximize total weight while connecting everything"	negate weights, run MST
"minimize the product of weights"	take logs, run MST
"which edges are in every MST / no MST?"	cut & cycle properties: an edge is in **every** MST iff it is the unique light edge across some cut; in **no** MST iff it is the strict maximum on some cycle
"the second-cheapest way to connect"	second-best MST — one edge swap away
"add junction points to save wire"	**Steiner tree** — NP-hard
"connect everything but no node may have degree > k"	**degree-constrained MST** — NP-hard
"cheapest *path* between two points"	**not MST** — that's shortest paths (M15). The MST path between `u` and `v` is the **minimax** path, not the shortest one

The most common confusion in this area: *the MST does not contain shortest paths.* The `u`–`v` path in an MST minimizes its **maximum** edge, not its **sum**. Skiena's Figure 8.1 makes the point visually: the geometric MST and the shortest-path tree from the center are visibly different trees on the same points.

Common Mistakes

MISTAKE	CONSEQUENCE
Using the MST to answer shortest-path queries	Wrong. MST paths are minimax, not minimum-sum
Applying weight negation to shortest paths ("longest path")	Fails — negative weights break Dijkstra, and longest simple path is NP-hard
Forgetting that Kruskal on a disconnected graph yields a **forest**	Silent wrong answer unless you check `|tree| == n−1`
Building an indexed heap with real `decrease-key` for Prim	Works, but the lazy-push version is shorter and the same asymptotically
Forgetting the `if (inTree[u]) continue;` stale-entry guard in lazy Prim	Vertices get added twice; wrong tree, wrong weight
No deterministic tie-break in Borůvka	Two components can select the same edge, or cycles of picks form
Assuming the MST is unique	Only guaranteed with distinct weights. All MSTs *do* share the same sorted weight list
Using `O(E lg V)` Prim on a dense graph	`O(V² lg V)` where `Θ(V²)` was available
Assuming a minimum-bottleneck tree is an MST	The implication runs one way only
Adding both directions of an undirected edge to Kruskal's edge list	Harmless but doubles the sort; and it will double-count if you sum without the union–find guard

---

Complexity Summary

ALGORITHM	TIME	SPACE	NOTES
Kruskal (sort + union–find)	$O(E \lg E) = O(E \lg V)$	$\Theta(V + E)$	sorting dominates; DSU part is $O(E\,\alpha(V))$
Kruskal with pre-sorted / integer-sortable edges	$O(E\,\alpha(V))$	$\Theta(V + E)$	radix sort the weights
Prim, array scan	$\Theta(V^2)$	$\Theta(V)$	best for dense graphs; no heap
Prim, binary heap (lazy)	$O(E \lg V)$	$\Theta(V + E)$	the practical default
Prim, Fibonacci heap	$O(E + V \lg V)$	$\Theta(V)$	theory; large constants
Borůvka	$O(E \lg V)$	$\Theta(V + E)$	$O(\lg V)$ rounds × $O(E)$; **parallelizable**
Karger–Klein–Tarjan	$O(V + E)$ **expected**	$\Theta(V + E)$	randomized; uses Borůvka + linear-time verification
Chazelle	$O(E\,\hat{\alpha}(E,V))$ deterministic	—	not greedy
MST verification (King)	$\Theta(V + E)$	$\Theta(V + E)$	"is this tree an MST?" is easier than finding one
Second-best MST	$O(V \cdot E \lg V)$ naive, $O(V^2)$ intended	$\Theta(V^2)$	one edge swap from the MST

One-Page Recall

- **Problem:** cheapest acyclic edge subset connecting all vertices. Every spanning tree has exactly $|V| - 1$ edges, so only weight matters.
- **`GENERIC-MST`:** maintain $A \subseteq$ some MST; repeatedly add a **safe** edge. $|V| - 1$ iterations.
- **Cut property (Theorem 21.1):** a **light edge** crossing **any cut that respects A** is **safe**. Proof: cut-and-paste — add (u,v) to an MST T, find the crossing edge (x,y) on the resulting cycle, swap. $w(T') = w(T) - w(x,y) + w(u,v) \le w(T)$.
- **Corollary 21.2:** the light edge joining a component of G_A to another component is safe. **This is the form both algorithms use.**
- **Cycle property:** the heaviest edge on any cycle is dispensable. Cut property says what to include; cycle property says what to exclude.
- **Kruskal:** sort edges ascending; add if the endpoints are in different components

(union–find). `O(E lg V)`. `A` is a **forest**. Better on **sparse** graphs. On a disconnected input it yields the minimum spanning **forest**.

- **Prim:** grow one tree from a root; repeatedly take the lightest edge leaving it (priority queue keyed by `key[v]` = cheapest edge to the tree). `θ(V²)` with an array (dense), `O(E lg V)` with a binary heap, `O(E + V lg V)` with a Fibonacci heap. Resembles **Dijkstra**.
- **Lazy decrease-key:** push a new `(key, v)` on every improvement and skip stale pops. Same `O(E lg V)`, much simpler. Reuse it in Dijkstra.
- **Borůvka (1926):** every component picks its cheapest outgoing edge each round; `O(lg V)` rounds. Needs a deterministic tie-break. **The parallelizable one**, and the basis of the near-linear and randomized-linear algorithms.
- **Uniqueness:** distinct weights ⟹ unique MST. In general, all MSTs share the same **sorted list** of edge weights.
- **Every MST is a minimum-bottleneck spanning tree.** The converse is false.
- **Transformations that work:** negate for **maximum** spanning tree; take logs for **minimum product**. Note MST is unusual in tolerating negation — shortest paths are not.
- **Transformations that don't: Steiner tree** (extra junctions) and **low-degree spanning tree** are both NP-hard.
- **MST ≠ shortest paths.** The `u`–`v` path in an MST minimizes the maximum edge, not the total.

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
MST on an implicit complete graph	1584 · Min Cost to Connect All Points	`n²` implicit edges, so **Prim's `O(V²)` beats Kruskal** — the density argument, made concrete
The cut and cycle properties, used	1489 · Find Critical and Pseudo-Critical Edges in MST	force an edge in / leave it out and re-run Kruskal; this is the cut property as an algorithm
Kruskal's engine on its own	684 · Redundant Connection · 1319 · Number of Operations to Make Network Connected	the union–find cycle test of line 7, with the sorting removed
Minimum **bottleneck**, not minimum sum	[778? use] [1631? use] — or [1102? use] — see the note below	every MST is a minimum-bottleneck spanning tree; the converse is false
Prim's structure, reused	743 · Network Delay Time	Dijkstra is Prim with one changed key (M15); write both and diff them
Components before connecting	547 · Number of Provinces	the disconnected case, where Kruskal yields a spanning **forest**

*(For minimum-bottleneck practice, the reliable route is CSES Problem Set → *Graph Algorithms* → "Road Construction" and neighbours, rather than a LeetCode number.)*

Beyond LeetCode. CSES Problem Set — *Graph Algorithms*. Codeforces `dsu` tag · `graphs` tag.

The drill that matters here: given a graph problem, ask *"is the objective the **sum** of chosen edges, or the **path** between two vertices?"* MST answers the first; shortest paths answers the second. Conflating them — expecting the MST path between `u` and `v` to be the shortest `u`–`v` path — is the single most common MST misconception, and it is false.

C++ Toolkit for This Module

*Language material from Weiss, **Data Structures and Algorithm Analysis in C++**, 4th ed., ch. 9 and §6.9.*

1. Sorting a struct: the comparator, or `operator<`

Kruskal sorts edges by weight. Two idioms, both fine:

```
struct Link { int u, v; long long w; };

void sortLinks(vector<Link>& e) {
    // (a) a lambda at the call site -- local, explicit, no
surprises
    sort(e.begin(), e.end(), [](const Link& a, const Link& b) {
return a.w < b.w; });
}

struct Link2 {
    int u, v; long long w;
    // (b) operator< as a member -- then sort(e.begin(), e.end())
just works,
    // and so do set<Link2>, map keys, and priority_queue<Link2>.
    bool operator<(const Link2& o) const { return w < o.w; }
};
```

Define `operator<` when the type has one obvious natural order; pass a lambda when the order is a property of *this algorithm* rather than of the type. An `Edge` has no natural order — it happens to be sorted by weight *here* — so the appendix passes a comparator.

2. Lazy decrease-key — the idiom this module and M15 share

`std::priority_queue` has **no `decrease-key`**, which `MST-PRIM` line 14 requires. The standard workaround:

```
void lazyIdiom() {
    using Item = pair<long long,int>;                       //
(key, vertex)
    priority_queue<Item, vector<Item>, greater<Item>> pq;   // MIN-
heap
    // On every improvement: PUSH A NEW ENTRY instead of updating
the old one.
    // On every pop: discard entries that are out of date.
    //     if (k > key[v]) continue;    // stale
}
```

The queue holds up to `E` entries instead of `V`, so the bound is `O(E lg E) = O(E lg V)` — **the same**, because `lg E ≤ lg V² = 2 lg V`. Memory grows from `Θ(V)` to `Θ(E)`. In exchange you never write a heap. **This is the right default**, and it reappears verbatim in Dijkstra.

The alternative — a `set<pair<long long,int>>` with `erase` then `insert` — gives a true decrease-key at `O(lg V)` and `Θ(V)` memory, with a much worse constant. Use it only when memory is genuinely tight.

3. `pair` ordering is lexicographic, which is exactly what you want here

`pair<long long,int>` compares `.first` first and only breaks ties on `.second`. So `priority_queue<pair<key,vertex>>` orders by key and breaks ties by vertex id — deterministic, reproducible, and free. **Put the key first.** Reversing to `pair<vertex,key>` silently orders by vertex id, which is a wrong answer that looks like a working program.

4. `Θ(V²)` Prim needs no heap at all

```
void densePrimShape(int n) {
    vector<long long> key(n, LLONG_MAX / 2);
    vector<char> inTree(n, 0);
    // V iterations, each doing a linear scan for the minimum:
    //   Theta(V) per extract-min x V extractions = Theta(V^2)
    //   plus Theta(1) per edge relaxation, Theta(E) total
    (void)key; (void)inTree;
}
```

For a **dense** graph (`E ≈ V²`) this beats the heap version: `Θ(V²)` against `O(E lg V) = O(V² lg V)`. The array version also has no allocation, no pointer chasing, and perfect locality. **On an implicit complete graph — LeetCode 1584 — it is simply the right algorithm.**

5. `long long` for accumulated weight

An MST sums `V−1` edge weights. With `V = 10⁵` and weights up to `10⁶`, the total is `10¹¹` — an `int` overflow, and signed overflow is undefined behaviour. Every total in the appendix is `long long`.

6. Returning several things from one function

```
struct MstOutput { long long weight; vector<int> edgeIds; bool
connected; };
```

A named struct beats `pair<long long, vector<int>>` — `.weight` and `.edgeIds` say what they are, and adding `connected` later does not break every call site (M03 toolkit §5).

Appendix — C++ for Every Pseudocode Block

```
// The representation every entry below uses.
struct MstEdge {
    int u = 0, v = 0;
    long long w = 0;
    int id = 0;              // position in the original input, for
reporting
};

struct MstGraph {
    int n = 0;
    vector<MstEdge> edges;                          // flat list --
Kruskal wants this
    vector<vector<pair<int,long long>>> adj;        // (neighbour,
weight) -- Prim wants this
    explicit MstGraph(int n_) : n(n_), adj(n_) {}
    void addEdge(int u, int v, long long w) {
        int id = (int)edges.size();
        edges.push_back({u, v, w, id});
        adj[u].push_back({v, w});                   // UNDIRECTED:
both directions
        adj[v].push_back({u, w});
    }
};

// The result of any MST algorithm here.
struct MstOutput {
    long long weight = 0;      // long long: V-1 weights can
overflow int (toolkit 5)
    vector<int> edgeIds;
```

```
    bool connected = false;    // false => this is a spanning
FOREST, not a tree
};
```

A1 GENERIC-MST

Pseudocode: Big Idea.

```
// GENERIC-MST is not runnable as written -- "find an edge that is
safe for A"
// is the entire problem. What it IS is the loop invariant that
both real
// algorithms maintain:
//
//     A is a subset of SOME minimum spanning tree.
//
// Every spanning tree of a connected graph has exactly |V| - 1
edges
// (Theorem B.2), so the loop runs exactly |V| - 1 times -- there
is no point
// minimising the NUMBER of edges, only their total weight.
//
// This version makes the invariant executable by taking the safe-
edge rule as
// a parameter. Kruskal and Prim are then just two arguments to it.
MstOutput genericMst(const MstGraph& g,
                     const function<int(const MstGraph&, const
vector<int>&)>& findSafeEdge) {
    MstOutput out;                                      // 1  A =
empty
    while ((int)out.edgeIds.size() < g.n - 1) {         // 2  while A
is not spanning
        int e = findSafeEdge(g, out.edgeIds);           // 3  find a
SAFE edge
        if (e < 0) break;                               //
disconnected: stop early
        out.edgeIds.push_back(e);                       // 4  A = A
union {(u,v)}
        out.weight += g.edges[e].w;
    }
    out.connected = ((int)out.edgeIds.size() == g.n - 1);
```

```
    return out;                                          // 5  return
A
}
```

The cut property (Theorem 21.1) is what makes any of this work.

> *Let `A` be a subset of some MST, let `(S, V–S)` be a cut that **respects** `A` (no edge of `A` crosses it), and let `(u,v)` be a **light** edge crossing that cut. Then `(u,v)` is **safe** for `A`.*

Proof (cut-and-paste / exchange). Let `T` be an MST containing `A`. If `(u,v) ∈ T`, done. Otherwise `T ∪ {(u,v)}` contains a unique cycle, and that cycle must cross the cut a second time at some edge `(x,y)`. Since `(u,v)` is light, `w(u,v) ≤ w(x,y)`. Form `T′ = T − {(x,y)} ∪ {(u,v)}`: it is a spanning tree, `w(T′) = w(T) − w(x,y) + w(u,v) ≤ w(T)`, so `T′` is also an MST — and it contains `A ∪ {(u,v)}`. ∎

Corollary 21.2 is the form both algorithms actually use: *if `C` is a connected component of the forest `G_A`, then the light edge joining `C` to another component is safe.* Kruskal applies it to whichever two components the next-cheapest edge joins; Prim applies it to the one component containing the root.

The cycle property is the mirror image: the **heaviest** edge on any cycle is never needed. The cut property tells you what to **include**; the cycle property tells you what to **exclude**.

A2 MST-KRUSKAL

Pseudocode: Part 2.

```
// Union-find, exactly as in M10 -- Kruskal is why that module
comes first.
class MstDisjointSet {
public:
    explicit MstDisjointSet(int n) : parent_(n), rank_(n, 0) {
        iota(parent_.begin(), parent_.end(), 0);      // 2-3  MAKE-
SET for every vertex
    }
    int find(int x) {
        while (x != parent_[x]) { parent_[x] = parent_[parent_[x]];
x = parent_[x]; }
        return x;                                          // path
halving
```

```
    }
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;                    // already
connected: a CYCLE
        if (rank_[ra] < rank_[rb]) swap(ra, rb);
        parent_[rb] = ra;
        if (rank_[ra] == rank_[rb]) ++rank_[ra];
        return true;
    }
private:
    vector<int> parent_, rank_;
};

MstOutput mstKruskal(const MstGraph& g) {
    MstOutput out;                                     // 1  A =
empty
    MstDisjointSet ds(g.n);                            // 2-3 MAKE-
SET(v) for all v

    // 4-5  sort the edges by increasing weight.
    // Sort a vector of INDICES, not of edges: an int is 4 bytes
and an MstEdge
    // is 24, so this moves 6x less memory, and it keeps g.edges
untouched so
    // `const MstGraph&` remains honest.
    vector<int> order(g.edges.size());
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(),
         [&g](int a, int b) { return g.edges[a].w < g.edges[b].w;
});

    for (int e : order) {                              // 6  in
sorted order
        const MstEdge& ed = g.edges[e];
        // 7  if FIND-SET(u) != FIND-SET(v)   -- unite() performs
the test AND the
        //    union, returning whether it merged. `false` means
this edge closes
        //    a cycle, which is exactly the cycle property in one
bit.
```

```
        if (ds.unite(ed.u, ed.v)) {                    // 8-9  A = A
+ edge; UNION(u,v)
            out.edgeIds.push_back(e);
            out.weight += ed.w;
            if ((int)out.edgeIds.size() == g.n - 1) break;   //
early exit: spanning
        }
    }
    out.connected = ((int)out.edgeIds.size() == g.n - 1);
    return out;                                        // 10 return
A
}
```

Complexity. `O(E lg E) = O(E lg V)` — the **sort dominates**. The union–find work is `O(E α(V))`, which is effectively linear (M10); the `lg` comes entirely from sorting. `E ≤ V²` gives `lg E ≤ 2 lg V`, which is why the two forms are the same. **Space** `Θ(V + E)`.

If the weights are small integers, radix-sort them and Kruskal becomes `O(E α(V))` — effectively linear. If the edges arrive already sorted, likewise.

`A` is a forest throughout, growing from `V` singleton trees to one. That is why Kruskal handles a **disconnected** input gracefully: it produces the minimum spanning **forest**, and `out.connected` reports which you got.

A3 MST-PRIM

Pseudocode: Part 3.

```
// PRIM with a binary heap and the LAZY decrease-key idiom (toolkit
2).
// `key[v]` is the weight of the cheapest known edge from the tree
to v -- CLRS's
// v.key -- and `parent[v]` is the other end of that edge, CLRS's
v.pi.
MstOutput mstPrim(const MstGraph& g, int r = 0) {
    MstOutput out;
    if (g.n == 0) return out;

    const long long INF = LLONG_MAX / 4;
    vector<long long> key(g.n, INF);                   // 1-2  u.key
= infinity
```

```
    vector<int> parent(g.n, -1);                        //      u.pi
= NIL
    vector<char> inTree(g.n, 0);                        //      "is u
still in Q?"
    key[r] = 0;                                         // 4  r.key =
0

    // 5  Q = all vertices. With the lazy idiom the queue holds
(key, vertex)
    //    pairs rather than vertices, and stale pairs are skipped
on pop.
    using Item = pair<long long,int>;
    priority_queue<Item, vector<Item>, greater<Item>> Q;   // MIN-
heap (toolkit 3)
    Q.push({0, r});

    while (!Q.empty()) {                                // 8  while Q
!= empty
        auto [k, u] = Q.top(); Q.pop();                 // 9  u =
EXTRACT-MIN(Q)
        if (inTree[u]) continue;                        //    a STALE
entry: skip it
        inTree[u] = 1;                                  //    add u
to the tree
        if (parent[u] != -1) { out.weight += k;
out.edgeIds.push_back(parent[u]); }

        for (const auto& [v, w] : g.adj[u]) {           // 10 for
each v in Adj[u]
            // 11 if v in Q and w(u,v) < v.key
            // `!inTree[v]` IS the "v in Q" test -- the queue
itself cannot
            // answer membership questions, so a separate array
does it.
            if (!inTree[v] && w < key[v]) {
                key[v] = w;                             // 13 v.key =
w(u,v)
                parent[v] = -1;                         //    (edge
id filled below)
                Q.push({w, v});                         // 14
DECREASE-KEY, lazily:
```

```
                //                                      //    push a
NEW entry, do not
                //                                      //    update
the old one
                // Remember which edge achieved this key, for
reporting.
                for (const MstEdge& e : g.edges)
                    if ((e.u == u && e.v == v) || (e.u == v && e.v
== u)) {
                        if (e.w == w) { parent[v] = e.id; break; }
                    }
            }
        }
    }
    out.connected = ((int)out.edgeIds.size() == g.n - 1);
    return out;
}

// PRIM WITH A LINEAR SCAN -- Theta(V^2), no heap, and the RIGHT
choice on a
// dense or implicit-complete graph (toolkit 4).
MstOutput mstPrimDense(const vector<vector<long long>>& w, int r =
0) {
    const int n = (int)w.size();
    const long long INF = LLONG_MAX / 4;
    MstOutput out;
    if (n == 0) return out;

    vector<long long> key(n, INF);
    vector<int> parent(n, -1);
    vector<char> inTree(n, 0);
    key[r] = 0;

    for (int iter = 0; iter < n; ++iter) {
        // EXTRACT-MIN by linear scan: Theta(V) here, Theta(V^2)
overall.
        int u = -1;
        long long best = INF;
        for (int i = 0; i < n; ++i)
            if (!inTree[i] && key[i] < best) { best = key[i]; u =
i; }
```

```
        if (u == -1) break;                              // remainder
is unreachable

        inTree[u] = 1;
        if (parent[u] != -1) out.weight += key[u];

        for (int v = 0; v < n; ++v)                      // relax
every neighbour:
            if (!inTree[v] && w[u][v] < key[v]) {        // Theta(V)
per vertex,
                key[v] = w[u][v];                        //
Theta(V^2) = Theta(E) overall
                parent[v] = u;
            }
    }
    int count = 0;
    for (int v = 0; v < n; ++v) if (parent[v] != -1) ++count;
    out.connected = (count == n - 1);
    return out;
}
```

Complexity — three implementations, and the choice is about density:

PRIORITY QUEUE	EXTRACT-MIN	DECREASE-KEY	TOTAL	BEST WHEN
array / linear scan	`O(V)`	`O(1)`	**`O(V²)`**	**dense**, or the graph is implicit
binary heap (lazy)	`O(lg V)`	`O(lg V)`	**`O(E lg V)`**	**sparse** — the practical default
Fibonacci heap	`O(lg V)` am.	`O(1)` am.	**`O(E + V lg V)`**	theory; large constants

Read the table as the formula `V · T_extract + E · T_decrease`. `V` extract-mins because each vertex joins the tree exactly once; `≤ E` decrease-keys because each edge can improve a key at most once from each end.

Prim and Kruskal are the same algorithm. Both instantiate `GENERIC-MST`; both are justified by Corollary 21.2. The only difference is the shape of `A`: **Kruskal keeps a forest** and joins whichever two components the cheapest remaining edge connects; **Prim keeps a single tree** and takes the cheapest edge leaving it.

And Prim is Dijkstra with one changed line. Prim ranks a candidate vertex by `w(u,v)` — the weight of the connecting edge. Dijkstra ranks it by `d[u] + w(u,v)` — the distance from the source. That is the entire difference, and it is also why Prim tolerates negative weights and Dijkstra does not: Prim's keys do not accumulate. See M15.

Next: M15 — Shortest Paths (CLRS 22–23 + Skiena 8.3–8.4) — relaxation and the shortest-path properties, Bellman-Ford, DAG shortest paths, Dijkstra, Floyd-Warshall and Johnson.